ASSIGNMENT 1: CLASSIFICATION AND PREDICTION

Carla Aranda 100523031 - Marina Juzgado 100523023

Phase 1. Instances collection

Original idea: before doing any previous classifiers or data exploring, we had to select the attributes to be used in the classification task. Originally we just picked the attributes we used in previous tutorials plus the position of the tail of the snake.

For this, we updated the original print_data_line function to add the desired attributes with simple Python code using the position of the vector created with all the positions of the body of the snake, a vector that gradually grows with time.

However, after testing the selected attributes to create a classifier and given a bad performance of it we changed the function again to add more data about the snake position, this being the first ten and last ten positions of the head and tail of the snake. For that, we created two vectors that take the first and last ten instances of the snake body, adding zeros in case the snake is shorter than desired.

```
# Extract the first 10 coordinates of the snake (from head)
 first_ten = game.snake_body[:10] + [(0, 0)] * (10 - len(game.snake_body))
 first_ten_flat = [str(coord) for pos in first_ten for coord in pos]
 # Extract the last 10 coordinates of the snake (from tail backwards)
 last_ten = game.snake_body[-10:] + [(0, 0)] * (10 - len(game.snake_body))
 last_ten_flat = [str(coord) for pos in last_ten for coord in pos]
```

Then, we added as attributes each instance of those vectors with a loop:

```
# Attributes for the first 10 snake positions
 for i in range(10):
 file.write(f"@ATTRIBUTE snake_x_{i + 1} NUMERIC\n")
 file.write(f"@ATTRIBUTE snake_y_{i + 1} NUMERIC\n")
 # Attributes for the last 10 snake positions
 for i in range(10):
 file.write(f"@ATTRIBUTE tail_x_{i + 1} NUMERIC\n")
 file.write(f"@ATTRIBUTE tail_y_{i + 1} NUMERIC\n")
```

In this way, we run the code and play the game manually until we reach 5000 number of instances so we have enough data examples to divide it in a proper training and test set. Yet before, as written in the statement, we wrote in each line the score of the following line until the last line as there is no next score we just wrote a 0. In this way, we modified the original data set of the snake with a Python function that adds the next score just before the direction:

```
def add_next_score_to_arff(file_path="all_data_snake.arff"):
 with open(file_path, "r") as file:
 lines = file.readlines()

 # Identify the line where list of attributes end and @Data starts
 attribute_index = next(i for i, line in enumerate(lines) if line.strip().upper() ==
"@DATA")

 # Move the new attribute between 'score' and 'direction'
 attribute_lines = lines[:attribute_index]
 for i, line in enumerate(attribute_lines):
 if line.strip().startswith("@ATTRIBUTE direction"):
 attribute_lines.insert(i, "@ATTRIBUTE next_score NUMERIC\n")
 break

 # Process data section
 data_lines = lines[attribute_index + 1:]
 updated_data = []

 for i in range(len(data_lines)):
 parts = data_lines[i].strip().split(",")
 if i < len(data_lines) - 1:
 next_score = data_lines[i + 1].split(",")[-2] # Obtain the score of the next line
 else:
 next_score = "0" # Last line reached

 # Insert next_score before the direction
 updated_line = ",".join(parts[:-1] + [next_score, parts[-1]])
 updated_data.append(f"{updated_line}\n")

 # Save the updated file
 with open(file_path, "w") as file:
 file.writelines(attribute_lines + ["@DATA\n"] + updated_data)
```

After all these changes, we generated a dataset with enough attributes for the classification, a line of the dataset looks like the following:

10,120,90,8,120,90,110,90,100,90,90,90,80,90,70,90,70,100,80,100,0,0,0,0,120,90,110,90,100,90,90,90,80,90,70,90,70,100,80,100,0,0,0,0,360,350,321,320,RIGHT

When we have the data set fully prepared, we divided it in training and test set, using 80% of the instances for training and the other 20% for test. First, we did a division just taking into account those percentages but not the order, just taking the first 80% and last 20% of the dataset but after a bad performance of the models in Phase 2, we decided to change it to add randomness, taking the same proportion of data lines but taking random lines.

```
input_file = "all_data_snake.arff"
 training_file = "training_keyboard.arff"
 test_file = "test_keyboard.arff"

 # Read the ARFF file
 with open(input_file, "r") as file:
 lines = file.readlines()

 # Find the header
 header_end = lines.index("@DATA\n")
 header = lines[:header_end + 1] # Include the '@DATA' line

 # Extract and shuffle data instances
 data_lines = lines[header_end + 1:]
 random.shuffle(data_lines) # Shuffle the data randomly

 # Split data into 80% training and 20% test
 split_index = int(len(data_lines) * 0.8)
 training_data = data_lines[:split_index]
 test_data = data_lines[split_index:]

 # Write training file
 with open(training_file, "w") as train_file:
 train_file.writelines(header + training_data)

 # Write test file
 with open(test_file, "w") as test_file:
 test_file.writelines(header + test_data)

 print(f"Training instances: {len(training_data)}")
 print(f"Test instances: {len(test_data)}")
```

Phase 2: Classification

First, with a simple Python function we changed the four directions (UP, DOWN, LEFT, RIGHT) into the ones asked in the statement (NORTH, SOUTH, WEST, EAST):

```
def replace_directions(input_file, output_file):
 replacements = {
 "UP": "NORTH",
 "DOWN": "SOUTH",
 "LEFT": "WEST",
 "RIGHT": "EAST"
 }

 with open(input_file, "r") as file:
 lines = file.readlines()

 # Find the beginning of data
 header_end = lines.index("@DATA\n")

 # Process just data lines
 updated_lines = [
 line.strip().rsplit(",", 1)[0] + "," + replacements.get(line.strip().rsplit(",", 1)
[1],
 line.strip().rsplit(",", 1)[1]) + "\n"
 if "," in line else line
 for line in lines[header_end + 1:]
 ]

 # Update the files
 with open(output_file, "w") as file:
 file.writelines(lines[:header_end + 1] + updated_lines)
```

As explained before, we first did a brief classification with a more reduced dataset yet the bad performance of the classifier lead us to change it. Analysing training dataset in the visualization part of Weka we immediately see the following:

- First of all see that all the directions (attribute we want to predict) are similarly distributed (between 1100 and 1250 out of 4723 instances)
- Then delete next_score as we can't use attributes related to the future.
- Some of the attributes are considered as continuous as the position of snake and food or the score

In the following table there are represented some of the performances of this original shorter dataset. We used all the variables without discretization, just with the continuous.

Classifier	Accuracy
ZeroR	27.6037%
J48	44.0305 %
Random Forest	48.0948 %
Naïve Bayes	53.0906 %

We can see that none of the accuracies above is significative. Performing the same after supervised and unsupervised discretization we observed with the confusion matrix that as it happens with ZeroR, all the classifiers like Random Tree and Part classified every single instance as SOUTH, which is the most popular of the four classes.

CHANGE OF DATA: adding the attributes explained in phase 1.

The method to divide the data in training and test set is the one provided on the explanation of Phase 1. Yet, before the random division of the data we used the data just in order, this leaded to a bad classification of the model, performing well with a 10 fold cross-validation of the training set when doing a Random Tree but when the test set was used to test the model the performance was much worse and the accuracy lowered drastically. This fact leaded us to guess that as cross validation divides the data in folds randomly and our division was not random, the classifier performed badly. Because of this, we finally divided the data in the way explained previously.

With the new dataset, the distribution in training set of the 4 types of directions we want to predict are the following:

Class	№ of instances
NORTH	3801
SOUTH	4001
WEST	4197
EAST	4488

The total number of instances is 16487 for the training set and 4122 for the test set (both with different data). The proportions between the four directions in both sets is practically the same, having a balanced data.

First of all, we removed the attribute next_score as it gives information about the future and it can't be used for prediction. We also deleted the attribute difficulty as it is always the same because we just played in the easy mode so it won't be useful for classification.

Using the continuous data without filters:

To begin with, we performance ZeroR, it is a simple and basic classifier that always predicts the most frequent variable (in our case EAST), achieving an accuracy of 26.0311 % with the following confidence matrix:

A	B	C	D	Classified as
0	0	0	950	**A = NORTH**
0	0	0	1023	**B = SOUTH**
0	0	0	1076	**C = WEST**
0	0	0	1073	**D = EAST**

It is the most basic and simple classifier that just counts the time each class appears in the training set and predicts the majority class for the test set. We have previously checked that EAST ha slightly more instances than the other variables. Despite it is not a good model to predict, it is a good reference model for future classifiers with other methods to consider if it is necessary a balance of the dataset in case other models have similar accuracy to ZeroR.

Trying other models like J48, Random Forest and Naïve Bayes we obtained the following results:

MODEL: **J48**, ACCURACY: **88,5978%,** CONFUSION MATRIX:

A	B	C	D	Classified as
830	12	49	59	**A = NORTH**
11	897	48	67	**B = SOUTH**
45	54	964	13	**C = WEST**
44	55	13	961	**D = EAST**

MODEL: **Random Forest**, ACCURACY: **92.5279%**, CONFUSION MATRIX:

A	B	C	D	Classified as
866	0	44	40	**A = NORTH**
0	944	33	46	**B = SOUTH**
39	32	1003	2	**C = WEST**
33	38	1	1001	**D = EAST**

MODEL: **Naïve Bayes**, ACCURACY: **45.4391%**, CONFUSION MATRIX:

A	B	C	D	Classified as
393	85	222	250	**A = NORTH**
86	418	182	337	**B = SOUTH**
170	232	476	198	**C = WEST**
185	151	151	586	**D = EAST**

MODEL: **PART**, ACCURACY: **89.8593%**, CONFUSION MATRIX:

A	B	C	D	Classified as
850	15	42	43	**A = NORTH**
11	923	40	49	**B = SOUTH**
56	40	967	13	**C = WEST**
45	53	11	964	**D = EAST**

We can see that J48, Random Forest and PART are good classifiers, meanwhile the accuracy of Naïve Bayes is not that good.

- J48 is the representation of C4.5 in Weka, it creates a decision tree by splitting data depending on the more informative attributes. It is effective when there are clear relationships between attributes and is able to use continuous data yet it only uses 1 tree.
- Random Forest creates multiple decision trees and each tree votes for a class for the classification, using the most common one. It avoids overfitting and is more robust and stable than J48 working well with many attributes and noise as it is the case in our dataset.
- Naïve Bayes performed worse than the other two as it assumes independence between attributes which in our case it is not true for example with all the positions of the body of the snake.
- PART is an algorithm that generates decision rules from a tree. Is a combination of J48 and decision rules making it easy to interpret and also efficient. It is more interpretable than a normal decision tree and less computationally expensive than J48 also avoiding over-pruning of leaves.
-

Apply unsupervised discretization:

In order to be able to perform a correct random Trees we discretize the data so the classifier handles it better.

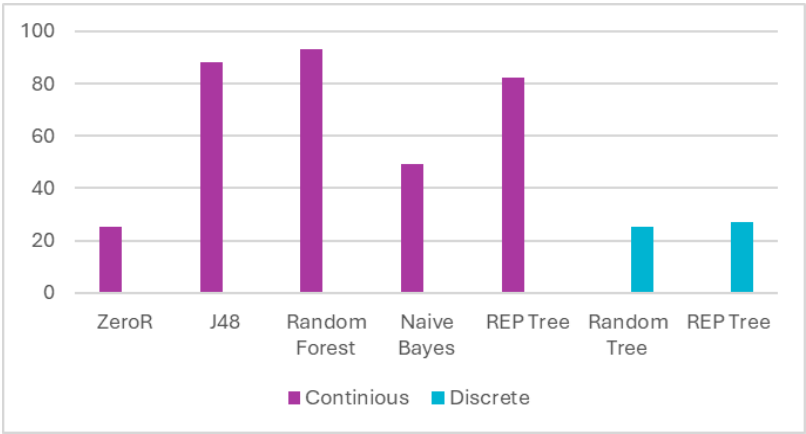
Random tree returns an accuracy of 26.0311 % which is quite low. This could be because of the loss of relevant information when discretizing, perhaps because we just used 10 bins for it. Trying the same with 20 bins to avoid loosing to much information we obtain the same accuracy.

To try to solve this first we visualize all the data, there are approximately the same distributed with enough values in each bin of every attribute. After a further analysis and given the previous very well performance of the Random Forest we conclude that it is due to a bad performance in the test set as Random Tree just uses one tree while Forest various.

The problem also persists using REP Tree (Reduced Error Pruning Tree) based in C4.5 that prunes the leaves to reduce overfitting, it returns an accuracy of 26.0311 % with discrete data while with continuous data it has an accuracy of 83.9641 % similar to J48. This leads to a single conclusion:

As every time we discretize the data the accuracy lowers considerably, it is better to use the original continuous data and not force discretization.

In the following graph there is a summary of all the classifier models we tried and its accuracy for a better visualization of the results:



At the same time, performing more filtering of the data like normalization and standardization the results present much less accuracy and worse performance of the classifier in the test set. This could be because the data is previously quite balanced in all the attributes, more or less being all the same except the coordinates of the tail and head as when it is inexistent, it is considered as 0 so when the snake is shorter it is considered as that.

To a better visualization the model, we performed all of those tests in Weka Experimenter, using the continuous data as it was the one that gave better accuracy results. In the following table, there are the results of the model created with Weka Experimenter using in all of them the training set:

Algorithm	J48	RandomForest	REPTree	ZeroR	NaiveBayes	PART
Accuracy	87.63	92.57	82.53	25.87	50.85	88.05

As seen previously in the Weka Explorer, again, the best algorithm to create a classifier is the Random Forest, followed by J48 and PART.

Phase 3: Building an automatic agent

To start with the third phase of the project, first we installed and checked all the programs that the statement require. Yet we had some problems with them:

1. **JDK:** as the laptop did not have java jdk we had to install the correct version (the one provided in aula global) and update the computer so that the path and JAVA_HOME of the system point to it. This was a rather confusing but necessary step as if it was not done, when trying to run the code in Python it gave us an error.

2. **Python:** at the same time, in order to be compatible with the functions of weka in wekal.py, Pycharm required Python 3.9 to be installed instead of the other version we had

3. **Python-weka-wrapper3**: installed without problems in the Pycharm terminal

4. **Javabridge:** in order to be able to install it as it is said in the second step of Phase 3, we had to use the terminal. However, it gave us a big error related with some virtual studio installer and as it was required in the terminal we installed it.

After all of those installations and processes, the code finally was run without problems and working perfectly. With some Debugs that after some investigation there were no bad signals and the game working in pygame.

In order to do it, we launched the functions provided in the statement and then updated the move function, specifically the same used in Tutorial 1 (move_tutorial_1). We named it move_snake and updated with the hints of the statement:

In this first part, we defined the x vector that will contain all of the information we need to the prediction, which is the same attributes used in training set and in the same order but without the ones deleted in weka.

```
def move_snake(game):
 # Obtain food and snake head positions
 food_x, food_y = game.food_pos
 snake_x, snake_y = game.snake_pos

 x = [
 snake_x, # x position of the head

 snake_y, # y position of the head
 len(game.snake_body), # snake length
 ]
 # Add first 10 parts of the snake
 for i in range(10):
 if i < len(game.snake_body):
 x.append(game.snake_body[i][0]) # x coord
 x.append(game.snake_body[i][1]) # y coord
 else:
 x.extend([0, 0]) # If snake smaller tan 10


 # Add last 10 parts of the snake
 if len(game.snake_body) < 10:
 for j in range(10):
 if j < len(game.snake_body):
 x.append(game.snake_body[j][0])
 x.append(game.snake_body[j][1])
 else:
 x.extend([0, 0])

 # Add food and score positions
 x.append(food_x) # food_x
 x.append(food_y) # food_y
 x.append(game.score) # score
```

Then, in the same function add the predictions that come from wekal.py class in other file as it is done in the following part of the python code where the predictions are printed for better visualization and then applied to the movement of the snake. In our case as it was seen previously in the results of the weka classifier, we selected a model based in Random Forest, represented in the variable “action” in the following code:

```
# Predict direction with Weka model
 action = weka.predict("./RandomForest.model",x, "./training_keyboard.arff")

# Print value of the prediction for visualization
 print("Predicción de Weka:", action) # Esto mostrará el valor de la predicción

 # Map the prediction
 if action == "NORTH":
 return "UP"
 elif action == "SOUTH":
 return "DOWN"
 elif action == "WEST":
 return "LEFT"
 elif action == "EAST":
 return "RIGHT"

 # If no table to predict
 return game.direction
```

Then, in the print_line_data, we deleted the direction as it is no longer necessary given that this phase of the project is based on the prediction.

Then when running the code, it is displayed the screen of the snake game. Now we are not able to move it with the keyboard as it is moved automatically based iion the predictions of weka as it is done in the code above.

First we tried the model with the best performance in Weka (Random Forest). Despite being the one that did best in weka, it did not perform well in practice as the snake was not moving at all. Because of that, we saved other models from weka that had also good performance and tried them in the game.

In this way, the second with highest accuracy was PART, trying it, some predictions were correct but not excellent as it focuses better on avoiding its own body than finding the fruit. Next, there are some conclusions that could explain a bad performance of the model:

- 20.000 instances could still be too little for a good performance. The screen that displays in the game is a square of 480x480. This means that there is an area of 230.400 to play in and clearly 20.000 instances would not show the decisions taken in all these squares, plus, we have to take into account that the snake has a body and there is a fruit so the possibilities are even higher.
- With just 20.000 instances, less than a 9% of the space is represented. This means that there are a lot of possibilities not evaluated by the model so when the snake reaches a new position of body and fruit combination, it does not know what to do.
- Not enough variety of data. A similar way of playing in each try to add data to the set could lead to little variation in possibilities and a single way of playing that the model would try to copy despite there are more possibilities.

To a further analysis, we can say that compared to the agent in Tutorial 1, this one show a worse performance with a lower score. This is because in Tutorial 1 we just programmed a python code so that the snake matches the coordinates of the fruit as it is done in the following code of Tutorial 1:

```
food_x, food_y = game.food_pos
 snake_x, snake_y = game.snake_pos

 #Move snake depending of the position of the fruit
 if food_x > snake_x and game.direction != "LEFT":
 return "RIGHT"
 if food_x < snake_x and game.direction != "RIGHT":
 return "LEFT"
 if food_y > snake_y and game.direction != "UP":
 return "DOWN"
 if food_y < snake_y and game.direction != "DOWN":
 return "UP"
 if food_x == snake_x and food_y > snake_y and game.direction == "UP":
 if snake_x < 240:
 return "RIGHT"
 else:
 return "LEFT"
 if food_x == snake_x and food_y < snake_y and game.direction == "DOWN":
 if snake_x <= 240:
 return "RIGHT"
 else:
 return "LEFT"
 if food_y == snake_y and food_x < snake_x and game.direction == "RIGHT":
 if snake_y < 240:
 return "DOWN"
 else:
 return "UP"
 if food_y == snake_y and food_x > snake_x and game.direction == "LEFT":
 if snake_y <= 240:
 return "DOWN"
 else:
 return "UP"
```

Here, the snake just reaches the coordinate of the fruit while in the model of this practice it tries to predict which movement the player would do in that situation, not taking into account the position of the fruit directly, just using it for the prediction as it uses not only that but other many information about the snake like its own body or the score.

Because of this, theorically, the model should perform better. However, because of the issues and possible reasons why it does not work better, it is not getting more score than the one in Tutorial 1.

Phase 4: Prediction

After some performing and new tries with the smart agent that finally manages to eat more than one fruit and has more movement, we obtained a new dataset composed by both keyboard and smart examples. We used the last dataset with more general data.

We implemented the function to add next_score as it is the attribute we want to predict, adding it in the last column of the new data lines after direction.

Ater this preprocessing, we will perform similar tests than in Phase 2 but now taking into account that we want to predict continuous data. The classifiers we used in the previous section to predict categorical data would not be useful to predict continuous ones as for example, the trees work much better with categorical data in order to divide in leaves and would not be useful to predict continuous values.

For that, we used some techniques like:

- **Linear Regression:** useful to predict linear data, however, as we also have discrete categorical predictors as all the directions, it would not be quite effective.

  1. MP5: recommended as it combines decision trees (for the discrete predictors) with linear regression (useful for the numerical predictors)

Correlation coefficient	0.9958
Mean absolute error	10.0176
Root mean squared error	77.8408
Relative absolute error	1.3584 %
Root relative squared error	9.1206 %

We see a correlation coefficient quite close to 1, this means a very high correlation between the model predictions and the real values. This means the MP5 performs quite well. On average, we see that the absolute error is a bit more than 10, this means that is has an error of 10 units in its predictions while the Root MSE tells that the model penalizes more big errors. The relative absolute error is quite low meaning the model is much better than just taking the mean od the data, this could be because a high variation in the predicted values. To finish, the RRSE is also low, which also implies a good model.

2. Multilayer perception: with a correlation coefficient of 0.9953 it also performs a good model. This model is a kind of neural network that learns complex relationships between attributes by transforming the discrete variables into numerical ones. The computational cost is a bit higher and with the previous good performance we would not need to do it. Besides, as it is shown in the table below, the Multilayer perception has not performed better than MP5 as it has a higher MAE, Root MSE, RAE and a similar RRSE, nothing to be extremely worried about but enough to classify the previous model as a better one.

Correlation coefficient	0.9953
Mean absolute error	26.4056
Root mean squared error	84.3909
Relative absolute error	3.5806 %
Root relative squared error	9.8881 %

3. SMOreg: Is a variant of SMO adapted to regression problems to predict continuous variables. It has performed well with a correlation coefficient of 0.9956 and very well overall errors, even better than the first model, which a very good MAE as its quite low. However, it has taken longer to process the training data with this method as it is a very effective but computationally expensive algorithm.

Correlation coefficient	0.9956
Mean absolute error	7.6643
Root mean squared error	79.6688
Relative absolute error	1.0393 %
Root relative squared error	9.3348 %

To sum up, those three methods as well as others like Random Forest which also has been tested again with a very good performance, are quite effective when predicting the next score. Overall, there have only been minimal variations in correlation coefficients and other kind of analysis, being all quite good to fit in our model. However, given the cost of some of them, it does not worth it a minimal upgrade in exchange for the cost as the models were doing quite well.

Questions

1. What is the difference between learning these models with instances coming from a human-controlled agent and an automatic one?

The main differences between the human-controlled agent and an automatic one are the quality of the collected data and the decision-making processes after that.

With a human-controlled agent, the collected data shows natural decision and intuition and strategic adjustments based on human experience and anticipation. Nevertheless, human movements may not be optimal due to hesitation or fatigue. These conditions will create a data set with different human strategies and some noise caused by non-optimal decisions.

An automatic agent uses a predefined rule set to make decisions during the game, ensuring consistency and usually the omptimal decision making. However, these models may not explore different approaches, which could lead to a less diverse data set.

In summary, a human-controlled agents provides diversity and adaptability to the data sets meanwhile automatic agents offers consistency and potentially more optimal decisions, a combination of both may result in a robust decision-making model.

2. If you wanted to transform the regression task into classification, what would you have to do? What do you think could be the practical application of predicting the score?

To transform the regression task into a classification problem, the continuous numerical values of the score would need to be converted into discrete categories. This can be done, for example, instead of using regression to estimate the score, using a binary classification to predict just if the score will increase or decrease.

The practical application of predicting the score could be game strategy optimization. The agent would be able to identify the movements that have an impact on the outcome of the score and improve the game strategy in order to obtain better score.

3. What are the advantages of predicting the score over classifying the action? Justify your answer.

Predicting the score offers advantages over directly classifying such as evaluate different actions based on expected outcomes instead of simply selecting a predefined action. Regression can be used for a more flexible decision-making process since it is not reduced to just two outcomes, as in binary classification. Moreover, score prediction allow to focus on long-term score increasing, instead of just follow a pattern that suggests increasing up to certain point, without knowing the causes.

4. Do you think that some improvement in the ranking could be achieved by incorporating an attribute that would indicate whether the score at the current time has dropped?

Yes, incorporating an attribute that indicates if the score has dropped could potentially improve the model's performance. When the agent knows that the score has dropped, it can adjust its behavior to avoid repeating the same mistake and also prioritize safer movements that are less likely to result in score reduction. Basically, it would be a beneficial step when it comes to optimize the decision-making process.

Conclusions

The technical development of this project has been based on testing different machine learning approaches to build a proper model for our purpose. Throughout the process, we were able to see how different data processing and machine learning techniques contribute to the performance of the models. After a long process of trial and error with the mentioned machine learning methods, we finally were able to find a model of automatic agent that optimizes the performance of our snake game .

The model that we obtained could be useful for a variety of real-world applications, depending on its specific purpose and functionality. If it was designed for a pattern recognition purpose, it could be applied in fields such as fraud detection or medical diagnosis. If it focuses more on automation, it may be a key mdoel for decision-making processes. The flexibility of machine learning models allows them to be adapted and improved for different scenarios, making them powerful tools for solving complex problems efficiently.

Moreover, the project development has also leaded to a series of difficulties and problems, both technical and more practical. Some of the problems he had encountered during the progress are the following:

- First of all, the main task has been the proper design of the intelligent agent in order to be able to properly predict the game movements. Both a too simple or a too complex model leaded to bad performances in practice, while a more complex model performs beter theoretically, in real world tasks it could end up being too specific for certain tasks. The opposite things happen with simpler models that performed better in practice than in theory. To solve this problem, a simple but complete model was the one opted for the final performance.
- Another important problem was the installation of diffrent programs to be able to use the javabridge package in Python. As the statement is not specific enough for every situation in each computer, some further investigation was necessary as each student had to install and configure different things depending on the laptop and operating system. It took too long to finally be able to connect Weka with Python to use the predictions, being this a setback for the proper development of the assignment.
- To finish with, a minor problem has been the proper configuration of the attributes in Python to use for the prediction as the same preprocessing than the one used in Weka must be done, this was at first slightly confusing and unspecific leading to some initial issues that were fastly solved after trying.

To conclude, after the problems presented during the development of this first practice and the ddifficulties encountered. A proper automatic agent was built as well as proper predictions in the different phases both in a better or a worse performance. Now we have learnt how to apply the models and classifiers built in Weka into practice and also how to select proper attributes that would help to predict in real world practices.